

Introduction to Data Science and Engineering

- The "tidyverse"



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

RB Luo / 羅銳邦

School of Computing and Data Science
University of Hong Kong

Some slides retrofitted by Michael Wu

Feb, 2023

TA Write R code to analyze the trend of COVID-19 in the U.S.

Here's some sample R code that could be used to analyze the trend of COVID-19 cases in the United States:

```
scss
Copy code

# Load required libraries
library(tidyverse)
library(readxl)

# Load data
covid_data <- read_excel("https://raw.githubusercontent.com/nytimes/covid-19-data/master/covid19.csv")

# Filter for U.S. data only
us_covid_data <- covid_data %>%
  filter(country == "US")

# Group data by date and sum cases
us_covid_data_grouped <- us_covid_data %>%
  group_by(date) %>%
  summarize(total_cases = sum(cases))

# Plot trend of total cases over time
ggplot(us_covid_data_grouped, aes(x = date, y = total_cases)) +
  geom_line() +
  ggtitle("Trend of COVID-19 Cases in the United States") +
  xlab("Date") +
  ylab("Total Cases")
```

This code first loads the required libraries, then loads COVID-19 data from the New York Times GitHub repository, filters the data to include only U.S. data, groups the data by date and sums the cases, and finally plots the trend of total cases over time.

Note that this code is just a sample to get you started and you may need to adjust it based on your specific data and analysis needs.

Jan, 2024

15 You Write R code to analyze the trend of COVID-19 in the U.S.

ChatGPT Certainly! To analyze the trend of COVID-19 in the U.S., you can use R and leverage libraries like `ggplot2` for data visualization and `dplyr` for data manipulation. Below is an example code using publicly available COVID-19 data from the "COVID-19 Data Repository by the Center for Systems Science and Engineering (CSSE) at Johns Hopkins University."

```
R
Copy code

# Install and load required packages
install.packages(c("tidyverse", "lubridate"))
library(tidyverse)
library(lubridate)

# Download COVID-19 data from Johns Hopkins University
url <- "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/covid19/data/countries.csv"
covid_data <- read_csv(url, stringsAsFactors = FALSE)

# Filter data for the United States
us_covid_data <- covid_data %>% filter(Country.Region == "US")

# Convert data to long format for time series analysis
us_covid_data_long <- us_covid_data %>%
  select(-Province.State, -Country.Region, -Lat, -Long) %>%
  pivot_longer(cols = starts_with("X"), names_to = "Date", values_to = "Cases")

# Plot the trend of COVID-19 cases in the U.S.
ggplot(us_covid_data_long, aes(x = Date, y = Cases, group = 1)) +
  geom_line(color = "blue") +
  labs(title = "COVID-19 Trend in the U.S.",
       x = "Date",
       y = "Number of Cases") +
  theme_minimal()
```

`lubridate`) before
`install.packages(c("tidyverse",`

August, 2025

ChatGPT 5

Share

Write R code to analyze the trend of COVID-19 in the U.S.

已思考 17s

Here's a clean, end-to-end R script that pulls public data, builds core trend metrics, and plots the U.S. COVID-19 trajectory. It uses *Our World in Data* for national trends (simple + stable) and optionally the CDC Socrata API for state-level detail.

```
r
Copy Edit

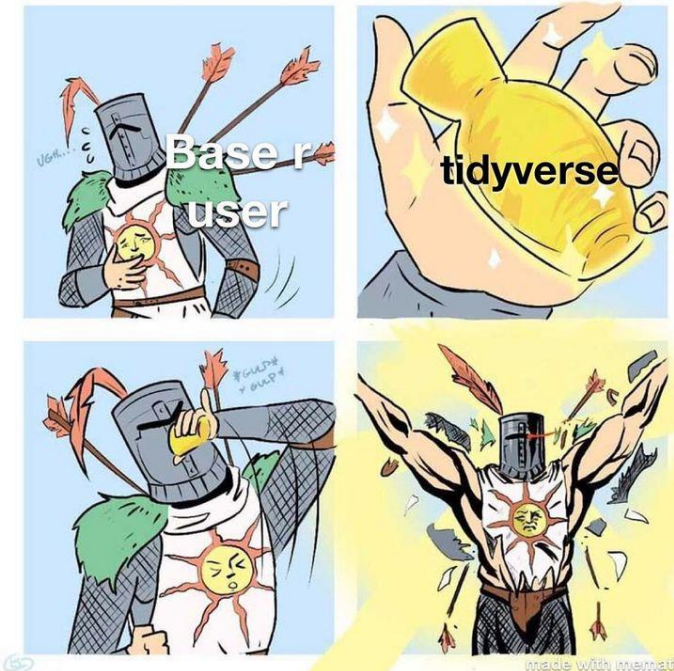
# ---- Setup ----
packages <- c("tidyverse", "lubridate", "scales", "zoo")
to_install <- setdiff(packages, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install, repos = "https://cloud.r-project.org")
invisible(lapply(packages, library, character.only = TRUE))

# ---- Load national data (Our World in Data) ----
# Source: https://ourworldindata.org/coronavirus
owid_url <- "https://covid.ourworldindata.org/data/owid-covid-data.csv"
us <- readr::read_csv(owid_url, show_col_types = FALSE) |>
  filter(location == "United States") |>
  select(date,
         population,
         new_cases,
         new_cases_smoothed,
         new_deaths,
         new_deaths_smoothed,
         new_tests,
         new_tests_smoothed,
         hosp_patients, # in hospital (snapshot), not all
         weekly_hosp_admissions, # per OWID (may be sparse)
         icu_patients,
         total_cases, total_deaths) |>
  mutate(
    cases_7dma = new_cases_smoothed,
    deaths_7dma = new_deaths_smoothed,
    tests_7dma = new_tests_smoothed,
    cases_per_100k_7dma = (cases_7dma / population) * 1e5,
    deaths_per_100k_7dma = (deaths_7dma / population) * 1e5,
    week = floor_date(date, "week", week_start = 1)
  )
```

ChatGPT being asked:
"Write R code to analyze the trend of COVID-19 in the U.S."

The tidyverse

In R basics, we learned manipulating vectors by reordering and subsetting them through indexing. However, once we start more advanced analyses, the preferred unit for data storage is not the vector but the data frame. In this chapter we learn to work directly with data frames, which greatly facilitate the organization of information. We will be using data frames for the remaining course. We will focus on a specific data format referred to as *tidy* and on specific collection of packages that are particularly helpful for working with *tidy* data referred to as the *tidyverse*.





Load the library

Ya, simple.

```
R 4.2.2 · ~/    
> library(tidyverse)  
— Attaching packages — tidyverse 1.3.2 —  
✓ ggplot2 3.4.0      ✓ purrr  1.0.0  
✓ tibble  3.1.8      ✓ dplyr  1.0.10  
✓ tidyr   1.2.1      ✓ stringr 1.5.0  
✓ readr   2.1.3      ✓ forcats 0.5.2  
— Conflicts — tidyverse_conflicts() —  
✗ dplyr::filter() masks stats::filter()  
✗ dplyr::lag()    masks stats::lag()
```



Tidy data

We say that a data table is in tidy format if each row represents one observation and columns represent the different variables available for each of these observations. The murders dataset is an example of a tidy data frame

```
R 4.2.2 · ~/ ↵
```

```
> library(dslabs)  
> data(murders)  
> murders
```

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65
7	Connecticut	CT	Northeast	3574097	97
8	Delaware	DE	South	897934	38
9	District of Columbia	DC	South	601723	99
10	Florida	FL	South	19687653	669

Tidy data

Which one is tidy?

```
> data(co2)
> co2
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
1959	315.42	316.31	316.50	317.56	318.13	318.00	316.39	314.65	313.68	313.18	314.66	315.43
1960	316.27	316.81	317.42	318.87	319.87	319.43	318.01	315.74	314.00	313.68	314.84	316.03
1961	316.73	317.54	318.38	319.31	320.42	319.61	318.42	316.63	314.83	315.16	315.94	316.85
1962	317.78	318.40	319.53	320.42	320.85	320.45	319.45	317.25	316.11	315.27	316.53	317.53
1963	318.58	318.92	319.70	321.22	322.08	321.31	319.58	317.61	316.05	315.83	316.91	318.20
1964	319.41	320.07	320.74	321.40	322.06	321.73	320.27	318.54	316.54	316.71	317.53	318.55
1965	319.27	320.28	320.73	321.97	322.00	321.71	321.05	318.71	317.66	317.14	318.70	319.25
1966	320.46	321.43	322.23	323.54	323.91	323.59	322.24	320.20	318.48	317.94	319.63	320.87
1967	322.17	322.34	322.88	324.25	324.83	323.93	322.38	320.76	319.10	319.24	320.56	321.80
1968	322.40	322.99	323.73	324.86	325.40	325.20	323.98	321.95	320.18	320.09	321.16	322.74

```
> co2_df <- data.frame(
+   Year = floor(time(co2)),
+   Month = cycle(co2),
+   CO2 = as.numeric(co2)
+ )
> co2_df
```

	Year	Month	CO2
1	1959	1	315.42
2	1959	2	316.31
3	1959	3	316.50
4	1959	4	317.56
5	1959	5	318.13
6	1959	6	318.00
7	1959	7	316.39
8	1959	8	314.65
9	1959	9	313.68
10	1959	10	313.18
11	1959	11	314.66
12	1959	12	315.43
13	1960	1	316.27
14	1960	2	316.81



Manipulating data frames

Adding a column with mutate

We want all the necessary information for our analysis to be included in the data table. So the first task is to add the murder rates to our murders data frame. The function `mutate` takes the data frame as a first argument and the name and values of the variable as a second argument using the convention `name = values`. So, to add murder rates, we do:

 R 4.2.2 · ~/ ↩

```
> library(dslabs)
> data("murders")
> murders <- mutate(murders, rate = total / population * 100000)
> head(murders)
```

	state	abb	region	population	total	rate
1	Alabama	AL	South	4779736	135	2.824424
2	Alaska	AK	West	710231	19	2.675186
3	Arizona	AZ	West	6392017	232	3.629527
4	Arkansas	AR	South	2915918	93	3.189390
5	California	CA	West	37253956	1257	3.374138
6	Colorado	CO	West	5029196	65	1.292453

Notice that here we used `total` and `population` inside the function, which are objects that are not defined in our workspace. But why don't we get an error?

This is one of `dplyr`'s main features. Functions in this package, such as `mutate`, know to look for variables in the data frame provided in the first argument. In the call to `mutate` above, `total` will have the values in `murders$total`. This approach makes the code much more readable

R 4.2.2 · ~/

```
> library(dslabs)
> data("murders")
> murders <- mutate(murders, rate = total / population * 100000)
> head(murders)
```

	state	abb	region	population	total	rate
1	Alabama	AL	South	4779736	135	2.824424
2	Alaska	AK	West	710231	19	2.675186
3	Arizona	AZ	West	6392017	232	3.629527
4	Arkansas	AR	South	2915918	93	3.189390
5	California	CA	West	37253956	1257	3.374138
6	Colorado	CO	West	5029196	65	1.292453

Subsetting with filter

Now suppose that we want to filter the data table to only show the entries for which the murder rate is lower than 0.71. To do this we use the filter function, which takes the data table as the first argument and then the conditional statement as the second.

R 4.2.2 · ~/

```
> filter(murders, rate <= 0.71)
```

	state	abb	region	population	total	rate
1	Hawaii	HI	West	1360301	7	0.5145920
2	Iowa	IA	North Central	3046355	21	0.6893484
3	New Hampshire	NH	Northeast	1316470	5	0.3798036
4	North Dakota	ND	North Central	672591	4	0.5947151
5	Vermont	VT	Northeast	625741	2	0.3196211

Selecting columns with select

Although the murders data table only has six columns, some data tables include hundreds. If we want to view just a few, we can use the dplyr select function. In the code below we select three columns, assign this to a new object and then filter the new object. In the call to select, the first argument murders is an object, but state, region, and rate are variable names

R 4.2.2 · ~/ ↻

```
> new_table <- select(murders, state, region, rate)
> filter(new_table, rate <= 0.71)
```

	state	region	rate
1	Hawaii	West	0.5145920
2	Iowa	North Central	0.6893484
3	New Hampshire	Northeast	0.3798036
4	North Dakota	North Central	0.5947151
5	Vermont	Northeast	0.3196211

Piping



#Rstats #Rlang

```
# R code in 1999
for(cyl in unique(mtcars[,2])){
  print(sum(mtcars[,2] == cyl))
}

# R code in 2007
table(mtcars$cyl)

# R code in 2014
mtcars %>%
  group_by(cyl) %>%
  tally()

# R code in 2021
mtcars |> subset(cyl == 4) |> head()

# R code in 2028
mtcars %>% summarise(
  count = sum(cyl == 4)
)

0.0 return()
```

In R we can perform a series of operations: add a new column, select some columns, filter observations, etc., by sending the results of one function to the subsequent one using the pipe operator

$$\% > \%$$

Piping almost always use the output of the left expression as the FIRST argument to the right expression.

```
> murders |> select(state, region, rate) |> filter(rate <= 0.71)
```

	state	region	rate
1	Hawaii	West	0.5145920
2	Iowa	North Central	0.6893484
3	New Hampshire	Northeast	0.3798036
4	North Dakota	North Central	0.5947151
5	Vermont	Northeast	0.3196211

Piping

```
R 4.2.2 · ~/ ↗  
> 16 |> sqrt()  
[1] 4  
> 16 |> sqrt() |> log2()  
[1] 2
```

is equivalent to

```
R 4.2.2 · ~/ ↗  
> sqrt(16)  
[1] 4  
> log2(sqrt(16))  
[1] 2
```

Pipe sends the result of the left statement to the right statement as the first argument

```
R 4.2.2 · ~/ ↗  
> 16 |> sqrt() |> log(base = 2)  
[1] 2
```

We can define other arguments as if the first argument is already defined

Where the idea of piping is from:

<https://www.geeksforgeeks.org/piping-in-unix-or-linux/>



Summarizing data (summarize)

An important part of exploratory data analysis is summarizing data. The average and standard deviation are two examples of widely used summary statistics. The `summarize` function in `dplyr` provides a way to compute summary statistics with intuitive and readable code. We start with a simple example based on heights. The `heights` dataset includes heights and sex reported by students in an in-class survey

```
R 4.2.2 · ~/    
> library(dplyr)  
> library(dslabs)  
> data(heights)  
> s <- heights |> filter(sex == "Female") |> summarize(average = mean  
(height), standard_deviation = sd(height))  
> s  
      average standard_deviation  
1 64.93942      3.760656
```

This takes our original data table as input, filters it to keep only females, and then produces a new summarized table with just the average and the standard deviation of heights. We get to choose the names of the columns of the resulting table. For example, above we decided to use `average` and `standard_deviation`, but we could have used other names just the same

Summarizing data (summarize)

For another example of how we can use the summarize function, let's compute the average murder rate for the United States. Remember that we have previously ran

```
R R 4.2.2 · ~/ ↵
```

```
> murders <- murders |> mutate(rate = total/population*100000)
```

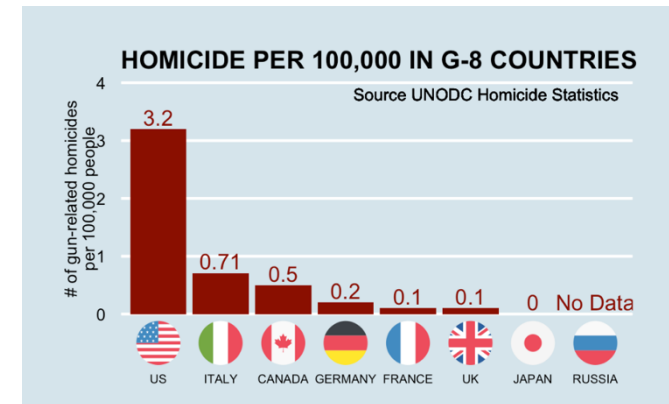
```
R R 4.2.2 · ~/ ↵
```

```
> summarize(murders, mean(rate))
```

```
mean(rate)
```

```
1 2.779125
```

Is this correct?



Summarizing data (summarize)

For another example of how we can use the summarize function, let's compute the average murder rate for the United States. Remember that we have previously ran

```
R 4.2.2 · ~/ ↵
```

```
> murders <- murders |> mutate(rate = total/population*100000)
```

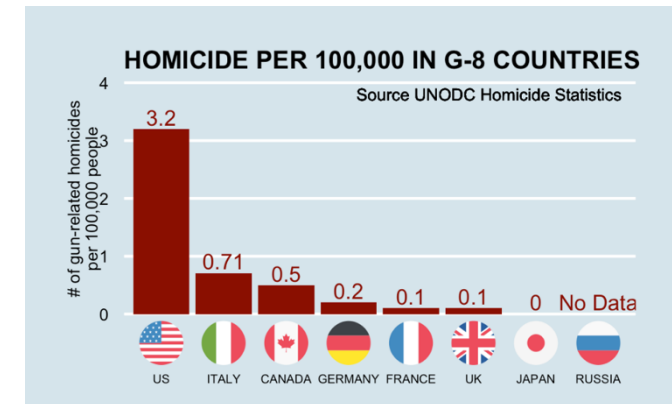
```
R 4.2.2 · ~/ ↵
```

```
> summarize(murders, mean(rate))  
mean(rate)  
1 2.779125
```

Is this correct?

```
R 4.2.2 · ~/ ↵
```

```
> us_murder_rate <- murders |> summarize(rate = sum(total) / sum(population) * 100000)  
> us_murder_rate  
rate  
1 3.034555
```



Rates cannot be averaged, to calculate the average murder rate, we should do this



Organize your command

Since the commands are getting longer, using Shift+Enter to split a command into multiple lines

```
R 4.2.2 · ~/ ↵  
> s <- heights |>  
+   filter(sex == "Female") |>  
+   summarize(average = mean(height), standard_deviation = sd(height))  
> s  
  average standard_deviation  
1 64.93942          3.760656
```

Multiple summaries

Suppose we want three summaries from the same variable such as the minimum, median, and maximum heights. The quantile function: `quantile(x, c(0, 0.5, 1))` returns the min (0th percentile), the median (50th percentile), and max (100th percentile) of the vector `x`. We can use it with `summarize` like this

```
R 4.2.2 · ~/ 
> heights |>
+ filter(sex == "Female") |>
+ summarize(min_median_max = quantile(height, c(0, 0.5, 1)))
  min_median_max
1      51.00000
2      64.98031
3      79.00000
```

However, notice that the summaries are returned in a row each. To obtain the results in different columns, we have to define a function that returns a data frame like this

```
R 4.2.2 · ~/ 
> min_median_max <- function(x){
+   qs <- quantile(x, c(0, 0.5, 1))
+   data.frame(min = qs[1], median = qs[2], max = qs[3])
+ }
> heights |>
+ filter(sex == "Female") |>
+ summarize(min_median_max(height))
  min  median max
1  51 64.98031  79
```

group_by then summarize

```
> heights |> group_by(sex)
# A tibble: 1,050 × 2
# Groups:   sex [2]
  sex    height
  <fct>   <dbl>
1 Male     75
2 Male     70
3 Male     68
4 Male     74
5 Male     61
6 Female    65
7 Female    66
8 Female    62
9 Female    66
10 Male     67
# ... with 1,040 more rows
# i Use `print(n = ...)` to see more rows
```

```
> data("heights")
> heights |>
+   group_by(sex) |>
+   summarize(n = n(), avg = mean(height), sd = sd(height))
# A tibble: 2 × 4
  sex      n  avg  sd
  <fct> <int> <dbl> <dbl>
1 Female  238  64.9  3.76
2 Male   812  69.3  3.61
```

A common operation in data exploration is to first stratify data into groups and then perform the same operations (e.g., summary statistics) on them separately. For example, compute average and standard deviation of heights for male and female separately.

The `group_by` function helps us do this: it transforms a data frame into a **grouped data frame**, and `dplyr` functions, in particular `summarize`, will behave separately for different groups.

Conceptually, you can think of a grouped data frame as many tables, with the same columns but non-overlapping rows. Use `n()` to get the count of different groups

group_by then summarize

```
> murders |>
+   group_by(region) |>
+   summarize(min_median_max(rate), n=n())
# A tibble: 4 × 5
  region      min median    max     n
  <fct>    <dbl>  <dbl> <dbl> <int>
1 Northeast 0.320    1.80  3.60     9
2 South     1.46    3.40 16.5    17
3 North Central 0.595    1.97  5.36    12
4 West      0.515    1.29  3.63    13
```

For another example, let's compute the minimum, median, and maximum murder rate in the four regions of US.

Sorting data frames

When examining a dataset, it is often convenient to sort the table by the different columns. We know about the `order` and `sort` function, but for ordering entire tables, the `dplyr` function `arrange` is useful. For example, here we order the states by population size

```
R 4.2.2 · ~/ ↗  
> murders |>  
+ arrange(population) |>  
+ head()
```

	state	abb	region	population	total	rate
1	Wyoming	WY	West	563626	5	0.8871131
2	District of Columbia	DC	South	601723	99	16.4527532
3	Vermont	VT	Northeast	625741	2	0.3196211
4	North Dakota	ND	North Central	672591	4	0.5947151
5	Alaska	AK	West	710231	19	2.6751860
6	South Dakota	SD	North Central	814180	8	0.9825837

```
R 4.2.2 · ~/ ↗  
> murders |>  
+ arrange(rate) |>  
+ head()
```

	state	abb	region	population	total	rate
1	Vermont	VT	Northeast	625741	2	0.3196211
2	New Hampshire	NH	Northeast	1316470	5	0.3798036
3	Hawaii	HI	West	1360301	7	0.5145920
4	North Dakota	ND	North Central	672591	4	0.5947151
5	Iowa	IA	North Central	3046355	21	0.6893484
6	Idaho	ID	West	1567582	12	0.7655102

With `arrange` we get to decide which column to sort by. To see the states by murder rate, from lowest to highest, we arrange by rate instead

Sorting data frames descendingly

Note that the default behavior is to order in ascending order. In dplyr, the function `desc` transforms a vector so that it is in descending order. To sort the table in descending order:

R 4.2.2 · ~/ ↗

```
> murders |>  
+ arrange(desc(rate)) |>  
+ head()
```

	state	abb	region	population	total	rate
1	District of Columbia	DC	South	601723	99	16.452753
2	Louisiana	LA	South	4533372	351	7.742581
3	Missouri	MO	North Central	5988927	321	5.359892
4	Maryland	MD	South	5773552	293	5.074866
5	South Carolina	SC	South	4625364	207	4.475323
6	Delaware	DE	South	897934	38	4.231937

Nested sorting

Sorting can be by multiple columns. Here we order by region, then within region we order by murder rate descendingly

```
R 4.2.2 · ~/   
> murders |>  
+ arrange(region, desc(rate)) |>  
+ head()  
      state abb  region population total   rate  
1  Pennsylvania PA Northeast  12702379  457 3.597751  
2    New Jersey NJ Northeast   8791894  246 2.798032  
3   Connecticut CT Northeast   3574097   97 2.713972  
4     New York NY Northeast  19378102  517 2.667960  
5 Massachusetts MA Northeast   6547629  118 1.802179  
6   Rhode Island RI Northeast   1052567   16 1.520093
```



The top n

`top_n` allows us to show the top n records according to a column without the need of sorting. This function takes a data frame as it's first argument, the number of rows to show in the second, and the variable to filter by in the third. Here is an example of how to see the top 5 rows:

```
R 4.2.2 · ~/   
> murders |> top_n(5, rate)
```

	state	abb	region	population	total	rate
1	District of Columbia	DC	South	601723	99	16.452753
2	Louisiana	LA	South	4533372	351	7.742581
3	Maryland	MD	South	5773552	293	5.074866
4	Missouri	MO	North Central	5988927	321	5.359892
5	South Carolina	SC	South	4625364	207	4.475323



The placeholder

```
R 4.2.2 · ~/ ↗  
> log(8, base = 2)  
[1] 3  
> 2 |> log(8, base = _)  
[1] 3
```

One of the advantages of using the pipe `|>` is that we do not have to keep naming new objects as we manipulate the data frame. The object on the left-hand side of the pipe is used as the first argument of the function on the right-hand side of the pipe. But what if we want to pass it as argument to the right-hand side function that is not the first? The answer is the placeholder operator `_`

A decorative graphic on the left side of the slide consisting of several overlapping, wavy, grey lines that create a sense of movement and depth.

The purr package

```
R 4.2.2 · ~/ 
> compute_s_n <- function(n){
+   x <- 1:n
+   sum(x)
+ }
> n <- 1:25
> s_n <- sapply(n, compute_s_n)
```

In the R basics, we learned `sapply` function, which enabled us to apply the same function to each element of a vector. We constructed a function and used `sapply` to compute the sum of the first `n` integers for several values of `n` like this

This type of operation, applying the same function or procedure to elements of an object, is quite common in data analysis. The `purrr` package includes functions similar to `sapply` but that better interact with other tidyverse functions. The main advantage is that we can better control the output type of functions. In contrast, `sapply` can return several different object types; for example, we might expect a numeric result from a line of code, but `sapply` might convert our result to character under some circumstances. `purrr` functions will never do this: they will return objects of a specified type or return an error if this is not possible

The purr package

 R 4.2.2 · ~/ ↗

```
> library(purrr)
> s_n <- map(n, compute_s_n)
> class(s_n)
[1] "list"
```

The first purrr function we will learn is `map`, which works very similar to `sapply` but always, without exception, returns a list

 R 4.2.2 · ~/ ↗

```
> s_n <- map_dbl(n, compute_s_n)
> class(s_n)
[1] "numeric"
```

If we want a numeric vector, we can instead use `map_dbl` which always returns a vector of numeric values. This produces the same results as the `sapply` call

 R 4.2.2 · ~/ ↗

```
> compute_s_n <- function(n){
+   x <- 1:n
+   data.frame(sum = sum(x))
+ }
> s_n <- map_df(n, compute_s_n)
```

A particularly useful purrr function for interacting with the rest of the tidyverse is `map_df`, which always returns a data frame. However, the function being called needs to return a vector or a list with names. For this reason, the `compute_s_n` function needs to be modified accordingly

Tidyverse conditionals (case_when)

A typical data analysis will often involve one or more conditional operations. In the R basics we learned the ifelse function, which we will use extensively in this book. Here we learn another two, case_when and between

R 4.2.2 · ~/

```
> x <- c(-2, -1, 0, 1, 2)
> case_when(x < 0 ~ "Negative",
+           x > 0 ~ "Positive",
+           TRUE ~ "Zero")
[1] "Negative" "Negative" "Zero"      "Positive" "Positive"
```

R 4.2.2 · ~/

```
> murders |>
+ mutate(group = case_when(
+   abb %in% c("ME", "NH", "VT", "MA", "RI", "CT") ~ "New England",
+   abb %in% c("WA", "OR", "CA") ~ "West Coast",
+   region == "South" ~ "South",
+   TRUE ~ "Other")) |>
+ group_by(group) |>
+ summarize(rate = sum(total) / sum(population) * 10^5)
# A tibble: 4 × 2
  group      rate
  <chr>    <dbl>
1 New England 1.72
2 Other      2.71
3 South      3.63
4 West Coast  2.90
```

The case_when function is useful for vectorizing conditional statements. It is similar to ifelse but can output any number of values, as opposed to just TRUE or FALSE. Here is an example splitting numbers into negative, positive, and 0

A common use for this function is to define categorical variables based on existing variables. For example, suppose we want to compare the murder rates in four groups of states: New England, West Coast, South, and other. For each state, we need to ask if it is in New England, if it is not we ask if it is in the West Coast, if not we ask if it is in the South, and if not we assign other. Here is how we use case_when to do this



Tidyverse conditionals (between)

```
R 4.2.2 · ~/   
> x >= a & x <= b
```

A common operation in data analysis is to determine if a value falls inside an interval. We can check this using conditionals. For example, to check if the elements of a vector *x* are between *a* and *b* we can do like this

```
R 4.2.2 · ~/   
> between(x, a, b)
```

However, this can become cumbersome, especially within the tidyverse approach. The `between` function performs the same operation

Python:

```
if 10000 <= number <= 30000:  
    pass
```